IIT (ISM) DHANBAD

Department of Computer Science & Engineering

PROJECT REPORT

Operating Systems (CSC211)

xv6 Modified System Calls: Signal Implementation

Submitted By:

Nayan Kumar Choudhary

Admission No: 23JE0636

Group No: G25

GitHub Repository:

https://github.com/darthvader323/G25_OS-Project

April 2026

1. Introduction

xv6 is a simple, Unix-like teaching operating system developed at MIT, implemented in ANSI C for the RISC-V multiprocessor architecture. It is widely used in operating systems courses to help students understand core OS concepts such as process management, system calls, memory management, and inter-process communication.

In this project, we extend the xv6 operating system by implementing a complete Signal System — a fundamental mechanism in Unix-like operating systems that allows processes to receive asynchronous notifications and handle them using custom user-defined functions.

Signals are one of the most important forms of Inter-Process Communication (IPC) in Unix systems. This implementation adds the ability for processes to register signal handlers, send signals to other processes, and safely return from signal handlers — all capabilities that the original xv6 lacks.

2. Objective

The primary objectives of this implementation are:

- Add a complete signal handling system to xv6 from scratch
- Implement sys_signal() syscall to register user-defined signal handlers
- Modify sys_kill() to support signal numbers (not just process termination)
- Implement sys_sigreturn() to safely restore process state after handler execution
- Add sys_sleep() syscall for user-space process sleeping
- Implement kernel-level signal delivery via deliver_signals() in the trap handler
- Demonstrate the complete signal flow with a working user-space test program

3. Features Implemented

3.1 signal(signum, handler) — Register Signal Handler

Allows a process to register a custom function to be called when a specific signal is received. Supports up to 32 different signal numbers (0-31), matching standard Unix signal conventions.

3.2 kill(pid, signum) — Send Signal to Process

Enhanced version of the original xv6 kill(). Now takes two arguments: the target process ID and a signal number. Uses a bitmask to mark the signal as pending in the target process without immediately terminating it.

3.3 sigreturn() — Restore State After Handler

A special syscall called at the end of every signal handler. Restores the CPU state (trapframe) that was saved before the handler was invoked, allowing the process to continue exactly where it was interrupted.

3.4 sleep(n) — User-Space Sleep

Puts the calling process to sleep for n timer ticks. Uses the kernel tick counter and condition variables for efficient sleeping without busy-waiting.

3.5 deliver_signals() — Kernel Signal Delivery

A kernel function called on every return from kernel to user space. Checks for pending signals, saves process state, and redirects execution to the registered signal handler.

4. Key Concepts Used

Concept	Explanation
Signal Bitmask	pending_signals integer where each bit represents one signal. Bit 10 set = signal 10 pending. Uses bitwise OR to set, AND NOT to clear.
Trapframe	Stores all CPU registers when entering kernel mode. We save/restore this to safely run signal handlers without corrupting process state.
Signal Delivery	Happens at usertrap() return — every time process goes back to user space, kernel checks pending signals and redirects to handler if needed.
sigreturn()	Special syscall that restores the saved trapframe, returning control to the exact instruction that was interrupted by the signal.
Handler Nesting Prevention	in_signal_handler flag prevents signals from being delivered while already inside a handler, avoiding stack overflow.
Default Action	If a signal arrives and no handler is registered, the process is killed (p->killed = 1) — same as standard Unix behavior.

5. Code Explanation — Files Modified

5.1 kernel/proc.h — Process Structure

Added four new fields inside struct proc to support signal handling:

```
void (*signal_handlers[32])(int); // 32 handler function pointers
int  pending_signals;             // bitmask of pending signals
struct trapframe signal_trapframe;// saved CPU state for sigreturn
int  in_signal_handler;           // 1 if currently in a handler
```

Why: Every process needs its own set of signal handlers and pending signal state. The bitmask allows 32 signals to be tracked in a single integer efficiently.

5.2 kernel/syscall.h — Syscall Numbers

Added new unique syscall numbers at the end of the file:

```
#define SYS_signal     23
#define SYS_sigreturn  24
#define SYS_sleep      25
```

Why: Every syscall needs a unique number. The kernel uses this number as an index into the syscall dispatch table to find which function to call.

5.3 kernel/syscall.c — Syscall Dispatch Table

Registered the new syscalls in the extern declaration list and the syscalls[] array:

```
extern uint64 sys_signal(void);
extern uint64 sys_sigreturn(void);
extern uint64 sys_sleep(void);

[SYS_signal]    sys_signal,
[SYS_sigreturn] sys_sigreturn,
[SYS_sleep]     sys_sleep,
```

Why: This dispatch table maps syscall numbers to their handler functions. Without this, the kernel cannot find the right function when a user program makes a syscall.

5.4 kernel/sysproc.c — Syscall Implementations

sys_signal() — Registers a signal handler:

```
uint64 sys_signal(void) {
  int signum; uint64 handler;
  argint(0, &signum);
  argaddr(1, &handler);
  myproc()->signal_handlers[signum] = (void(*)(int))handler;
  return 0;
}
```

sys_sigreturn() — Restores state after handler:

```
uint64 sys_sigreturn(void) {
  struct proc *p = myproc();
  *p->trapframe = p->signal_trapframe;
  p->in_signal_handler = 0;
  return 0;
}
```

sys_kill() — Modified to accept signal number:

```
uint64 sys_kill(void) {
  int pid, signum;
  argint(0, &pid);
  argint(1, &signum);
  return kkill(pid, signum);
}
```

5.5 kernel/proc.c — Modified kkill()

Changed kkill() from one argument to two, now sets signal bitmask instead of immediately killing:

```
int kkill(int pid, int signum) {
  // find the process
  p->pending_signals |= (1 << signum); // set signal bit
  if(p->state == SLEEPING)
    p->state = RUNNABLE;   // wake up sleeping process
  return 0;
}
```

Why: Old kill() immediately terminated a process. New kkill() marks a signal as pending using bitwise OR, allowing the process to handle it gracefully via its registered handler.

5.6 kernel/trap.c — Signal Delivery

Added deliver_signals() function and called it at the end of usertrap(), just before returning to user space:

```
void deliver_signals(void) {
  struct proc *p = myproc();
  if(p->in_signal_handler) return;  // prevent nesting
  for(int sig = 0; sig < 32; sig++) {
    if(p->pending_signals & (1 << sig)) {
      if(p->signal_handlers[sig] != 0) {
        p->pending_signals &= ~(1 << sig); // clear bit
        p->signal_trapframe = *p->trapframe;// save state
        p->in_signal_handler = 1;
        p->trapframe->a0 = sig;             // pass signum
        p->trapframe->epc = (uint64)p->signal_handlers[sig];
        break;
      } else { p->killed = 1; }  // default: kill
    }
  }
}
```

Why usertrap(): This function is called on every syscall, interrupt, and exception — it is the gateway back to user space. Checking here ensures signals are always delivered promptly and safely.

5.7 user/signaltest.c — Demo Program

Created a new user-space program that demonstrates the complete signal flow:

```
signal(15, sigterm_handler);   // register SIGTERM handler
signal(10, sigusr1_handler);   // register SIGUSR1 handler
int pid = fork();              // create child process
// child sleeps, parent sends signals to child
kill(pid, 10);  // send SIGUSR1
kill(pid, 15);  // send SIGTERM
// child handlers fire and print messages
```

6. Complete Signal Flow Diagram

The following describes the complete lifecycle of a signal in our implementation:

- Step 1: User calls signal(10, my_handler) -> sys_signal() stores function pointer in proc->signal_handlers[10]
- Step 2: Another process calls kill(pid, 10) -> sys_kill() -> kkill() sets bit 10 in target proc->pending_signals
- Step 3: Target process makes any syscall or gets a timer interrupt -> usertrap() runs
- Step 4: Before returning to user space, usertrap() calls deliver_signals()
- Step 5: deliver_signals() finds bit 10 set, saves current trapframe to signal_trapframe
- Step 6: Sets trapframe->epc to address of my_handler, sets a0=10 (signal number)
- Step 7: Process returns to user space and runs my_handler(10) instead of original code
- Step 8: my_handler calls sigreturn() -> sys_sigreturn() restores original trapframe
- Step 9: Process continues from exactly where it was before the signal arrived

7. Summary of All Changes

File	Changes Made
kernel/proc.h	Added 4 fields to struct proc: signal_handlers[], pending_signals, signal_trapframe, in_signal_handler
kernel/syscall.h	Added SYS_signal=23, SYS_sigreturn=24, SYS_sleep=25
kernel/syscall.c	Added extern declarations and dispatch table entries for 3 new syscalls
kernel/sysproc.c	Implemented sys_signal(), sys_sigreturn(), sys_sleep(). Modified sys_kill() to take 2 args
kernel/proc.c	Modified kkill() to take signal number, use bitmask instead of killing immediately
kernel/trap.c	Added deliver_signals() function, called it in usertrap() before prepare_return()
kernel/defs.h	Updated kkill() declaration from int(int) to int(int,int)
user/user.h	Added declarations: signal(), sigreturn(), sleep(), updated kill() to take 2 args
user/usys.pl	Added entry stubs: signal, sigreturn, sleep
user/signaltest.c	NEW FILE: Complete demo program showing signal registration, sending, and handling
user/kill.c	Updated kill() call to pass signal number (15=SIGTERM) as second argument

user/usertests.c	Updated all kill() calls to pass signal number as second argument
user/grind.c	Updated all kill() calls to pass signal number as second argument
Makefile	Added _signaltest to UPROGS list

8. Execution Steps

8.1 Setup & Build

```
# Step 1: Clone the repository
git clone https://github.com/darthvader323/G25_OS-Project.git
cd G25_OS-Project

# Step 2: Install dependencies (Ubuntu)
sudo apt install -y git build-essential qemu-system-misc
sudo apt install -y gcc-riscv64-linux-gnu binutils-riscv64-linux-gnu

# Step 3: Build and run xv6
make qemu
```

8.2 Running the Signal Demo

```
# Inside xv6 shell, type:
$ signaltest
```

8.3 Expected Output

```
=== xv6 Signal Demo ===
Child PID=4: registering handlers and sleeping...
Parent: sending SIGUSR1 (10) to child PID=4
Parent: sending SIGTERM (15) to child PID=4
Child: SIGUSR1 (10) received!
=== Signal Demo Complete ===
```

8.4 Exiting xv6

```
# Press Ctrl+A then X to exit QEMU
```

9. Screenshots

```
xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh

[Kernel] Process Created: Parent PID 1 -> Child PID 2
$ signaltest

[Kernel] Process Created: Parent PID 2 -> Child PID 3

=== xv6 Signal Demo ===

[Kernel] Process Created: Parent PID 3 -> Child PID 4
Child PID=4: registering handlers and sleeping...
Parent: sending SIGUSR1 (10) to child PID=4
Parent: sending SIGTERM (15) to child PID=4
Child: SIGUSR1 (10) received!
=== Signal Demo Complete ===

$ |
```

10. Conclusion

In this project, we successfully implemented a complete signal handling system for the xv6 operating system. This involved modifying the core kernel files including the process structure, trap handler, and syscall infrastructure, as well as adding new user-space interfaces.

The implementation demonstrates a deep understanding of how Unix-like operating systems handle asynchronous events through signals. Key learning outcomes include:

- Understanding of the xv6 kernel architecture and syscall mechanism
- Implementation of signal bitmasks for efficient pending signal tracking
- Safe signal delivery by saving and restoring CPU state (trapframe)
- The role of usertrap() as the kernel-user space gateway
- Prevention of signal handler nesting using state flags
- User-space signal handler registration and execution flow

The signal system implemented here closely mirrors how real Unix/Linux systems handle signals, making this a valuable practical exercise in operating systems development. The working demo (signaltest) confirms that signals can be sent between processes and handled correctly using user-defined functions.

11. References

- xv6 Book: https://pdos.csail.mit.edu/6.828/2023/xv6/book-riscv-rev3.pdf
- MIT 6.1810 Operating System Engineering: https://pdos.csail.mit.edu/6.1810/
- xv6 Source Code: https://github.com/mit-pdos/xv6-riscv
- Group Repository: https://github.com/darthvader323/G25_OS-Project
- Linux Signal Manual: man 7 signal